

Lecture 20

State Reduction and Sequential Design

Equivalent states · state reduction · state assignment · designing with D, JK and T flip-flops · unused states



Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 5, §5.7–5.8

Today's roadmap

1

§5.7

State reduction

Merge equivalent states

2

§5.8

State assignment

Binary, Gray or one-hot

3

§5.8

Design with D

Table → maps → circuit

4

§5.8

JK and T design

Excitation tables

By the end of class you can...

- Find equivalent states by row matching and the implication table
- Choose a state assignment and count flip-flops
- Design a sequence detector with D flip-flops
- Use excitation tables to design with JK and T flip-flops
- Check that unused states cannot trap the circuit

Agenda for today's session



Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap	—
0:05 – 0:30	Equivalent states; implication table	Example 1
0:30 – 0:40	State assignment	—
0:40 – 0:50	Break	
0:50 – 1:15	The design procedure with D flip-flops	Example 2
1:15 – 1:40	Excitation tables; JK and T design	Examples 3, 4 · Your turn
1:40 – 1:50	Unused states; concept check; wrap-up	—

Equivalent states

§5.7

Definition

Two states are equivalent when, for every input, they give the same output and go to equivalent next states.

Why reduce?

Fewer states can mean fewer flip-flops ($2^n \geq \text{states}$) and simpler logic, with identical input-output behavior.

Two methods

Row matching: merge identical rows, repeat.

Implication table: finds every equivalence.

Row matching

1. Look for two rows with the same next states and outputs.
2. Keep one; replace the other everywhere in the table.
3. Repeat: merging can make new rows identical.

Stops when no rows match; the implication table may still find more.

Example 1 — Reduce a state table

§5.7

EXAMPLE

Reduce this seven-state table. Columns: next state and output for $x = 0$ and $x = 1$.

State	Next $x=0$	Next $x=1$	y $x=0$	y $x=1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

State	Next $x=0$	Next $x=1$	y $x=0$	y $x=1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

Reduced: five states

Equivalent pairs: $e \equiv g$, then $d \equiv f$. Every step is on the next slide.

Example 1 — Step by step

§5.7

- 1 Equivalent: same outputs, and equivalent next states for each x
- 2 Group by outputs: $\{a, b, c\}$ give 0, 0; $\{d, e, f, g\}$ give 0, 1
- 3 In the second group: $e \rightarrow (a, f)$ and $g \rightarrow (a, f)$: identical
- 4 So $e \equiv g$. Replace g by e everywhere
- 5 Now $f \rightarrow (e, f)$ and $d \rightarrow (e, f)$: identical
- 6 So $d \equiv f$. Replace f by d : $d \rightarrow (e, d)$, $e \rightarrow (a, d)$
- 7 Check the first group: $a \rightarrow (a, b)$, $b \rightarrow (c, d)$, $c \rightarrow (a, d)$ differ
- 8 Repeat until no rows match: nothing more merges
- 9 Result: $\{a\}, \{b\}, \{c\}, \{d, f\}, \{e, g\}$: **five states**
- 10 Still 3 flip-flops, but 3 unused codes become don't-cares

The implication table

§5.7

	a	b	c	d	e	f
b	X					
c	X	X				
d	X	X	X			
e	X	X	X	X		
f	X	X	X	eg	X	
g	X	X	X	X	✓	X

Procedure

One cell per pair of states.

X if the outputs differ.

Otherwise list the implied pairs (next states that must also be equivalent); ✓ if none.

Pass again: X any cell whose implied pair is X. Repeat.

Unmarked cells are equivalent: here (d, f) and (e, g) .

State assignment

§5.8

State	Binary	Gray	One-hot
a	000	000	00001
b	001	001	00010
c	010	011	00100
d	011	010	01000
e	100	110	10000

Trade-offs

Binary: fewest flip-flops, $\lceil \log_2 m \rceil$.

Gray: states that follow each other differ in one bit; K-map groups tend to be larger.

One-hot: one flip-flop per state; next-state logic is trivial and fast (common in FPGAs).

Unused codes (3 here in binary) are don't-cares.

Break

10 minutes — we resume at the 0:50 mark

Next: designing sequential circuits

The design procedure

§5.8

- 1 From the word description, draw the state diagram.
- 2 Reduce the number of states if possible.
- 3 Assign binary codes: n flip-flops for up to 2^n states.
- 4 Choose the flip-flop type (D is the simplest to design with).
- 5 Build the encoded state table: present, input, next, output.
- 6 Derive and simplify each flip-flop input and output equation.
- 7 Draw the circuit; check any unused states.

Flip-flop inputs

D: $D = Q(t + 1)$, read straight from the table.

JK: look up J and K in the excitation table.

T: $T = Q \oplus Q(t + 1)$.

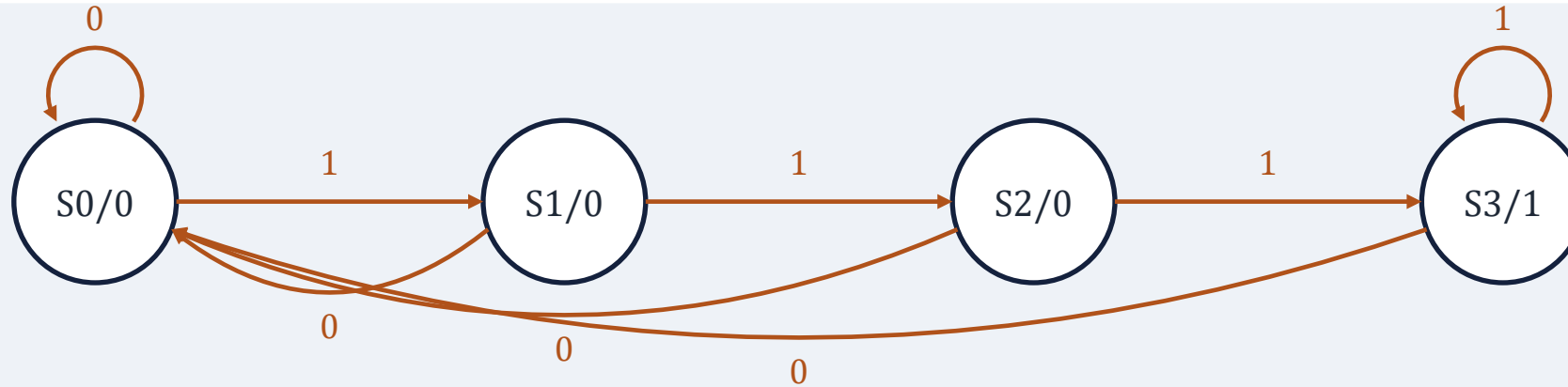
Unused states and excitation X entries are don't-cares.

Example 2 — A sequence detector

§5.8

EXAMPLE

Output $y = 1$ after three or more consecutive 1s on input x . Use D flip-flops.

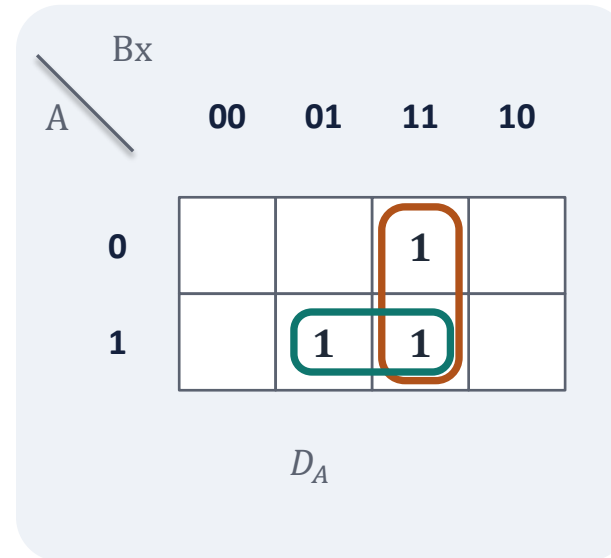


- 1 State S_k : the last k inputs were 1 ($k \leq 3$)
- 2 From any state, $x = 0 \rightarrow$ back to S_0
- 3 $x = 1 \rightarrow S_{k+1}$; S_3 stays in S_3
- 4 Moore: $y = 1$ only in S_3
- 5 Four states $\rightarrow$ two flip-flops A, B
- 6 Codes: $S_0 = 00, S_1 = 01, S_2 = 10, S_3 = 11$

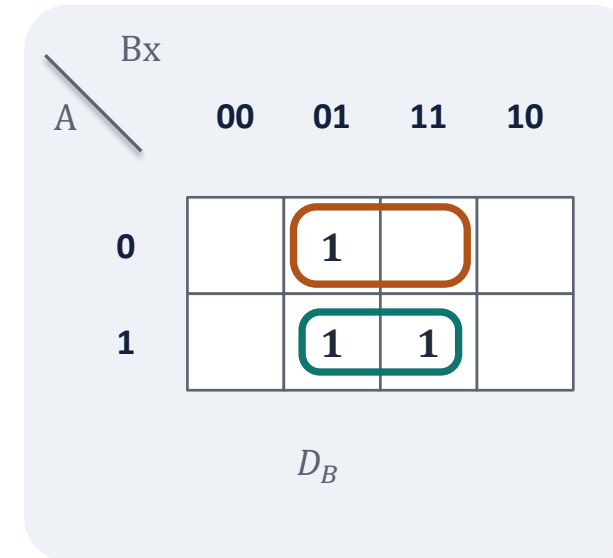
Example 2 — Table and maps

§5.8

A B	x	A ⁺ B ⁺	y
0 0	0	0 0	0
0 0	1	0 1	0
0 1	0	0 0	0
0 1	1	1 0	0
1 0	0	0 0	0
1 0	1	1 1	0
1 1	0	0 0	1
1 1	1	1 1	1



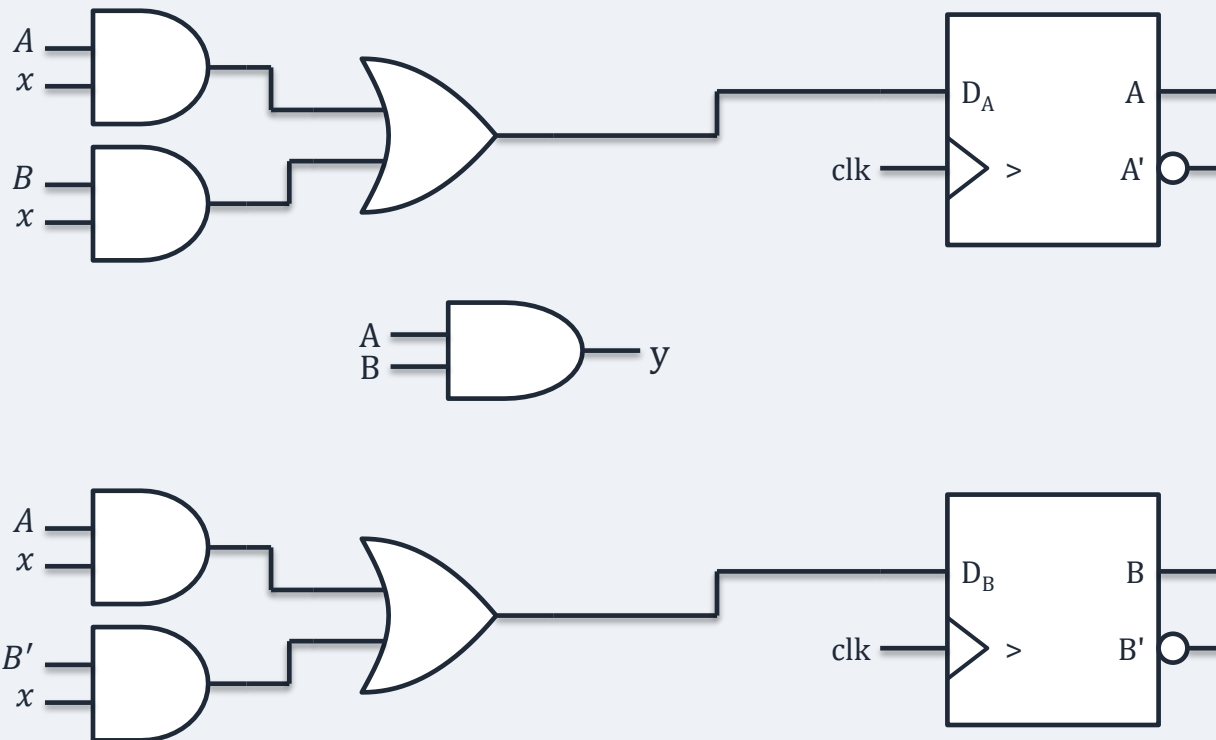
- 1 Next states from the diagram
- 2 D flip-flops: $D_A = A^+$, $D_B = B^+$
- 3 D_A : m3, m7 $\rightarrow Bx$; m5, m7 $\rightarrow Ax$



- 4 $D_A = Ax + Bx$
- 5 D_B : m1, m5 $\rightarrow B'x$; m5, m7 $\rightarrow Ax$: $D_B = Ax + B'x$
- 6 $y = 1$ only in 11: $y = AB$

Example 2 — The circuit

§5.8



Check a sequence

$x = 0\ 1\ 1\ 1\ 1\ 0$

state: $00 \rightarrow 00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 11 \rightarrow 00$

$y = 0\ 0\ 0\ 0\ 1\ 1\ 0$

Feedback signals A, B, B' are drawn as labels to keep the figure readable.

Excitation tables

§5.8

$Q \rightarrow Q^+$	J	K
$0 \rightarrow 0$	0	X
$0 \rightarrow 1$	1	X
$1 \rightarrow 0$	X	1
$1 \rightarrow 1$	X	0

$Q \rightarrow Q^+$	T
$0 \rightarrow 0$	0
$0 \rightarrow 1$	1
$1 \rightarrow 0$	1
$1 \rightarrow 1$	0

$Q \rightarrow Q^+$	D
$0 \rightarrow 0$	0
$0 \rightarrow 1$	1
$1 \rightarrow 0$	0
$1 \rightarrow 1$	1

Reading an excitation table

It answers: what input makes the flip-flop go from Q to Q^+ ? For JK, each transition has two ways to happen, so one input is a don't-care: JK designs usually need the fewest gates.

Example 3 — Design with JK flip-flops

§5.8

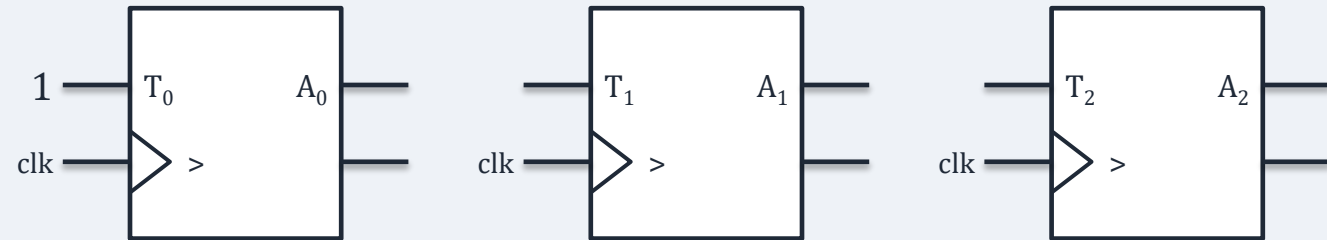
A B	x	A ⁺ B ⁺	J _A	K _A	J _B	K _B
0 0	0	0 0	0	X	0	X
0 0	1	0 1	0	X	1	X
0 1	0	1 0	1	X	X	1
0 1	1	0 1	0	X	X	0
1 0	0	1 0	X	0	0	X
1 0	1	1 1	X	0	1	X
1 1	0	1 1	X	0	X	0
1 1	1	0 0	X	1	X	1

- 1 Write each present state, input and next state
- 2 Excitation: $0 \rightarrow 0: J = 0, K = X$; $0 \rightarrow 1: J = 1, K = X$
- 3 $1 \rightarrow 0: J = X, K = 1$; $1 \rightarrow 1: J = X, K = 0$
- 4 Fill J_A, K_A from $A \rightarrow A^+$; J_B, K_B from $B \rightarrow B^+$
- 5 Map each column; every X is a don't-care
- 6 $J_A = Bx', K_A = Bx$
- 7 $J_B = x, K_B = A'x' + Ax$
- 8 $K_B = A'x' + Ax = (A \oplus x)',$ one XNOR

Example 4 — Counter with T flip-flops

§5.8

$A_2A_1A_0$	next	T_2	T_1	T_0
000	001	0	0	1
001	010	0	1	1
010	011	0	0	1
011	100	1	1	1
100	101	0	0	1
101	110	0	1	1
110	111	0	0	1
111	000	1	1	1



$$T_0 = 1, \quad T_1 = A_0, \quad T_2 = A_1A_0$$

- 1 $T = Q \oplus Q^+$: 1 where the bit changes
- 2 T_0 is 1 on every row $\rightarrow T_0 = 1$
- 3 $T_1 = 1$ on 001, 011, 101, 111 $\rightarrow T_1 = A_0$
- 4 $T_2 = 1$ on 011, 111 $\rightarrow T_2 = A_1A_0$
- 5 A bit toggles when every lower bit is 1
- 6 Counts 000 $\rightarrow$ 001 $\rightarrow$... $\rightarrow$ 111 $\rightarrow$ 000

Your turn — Counter with enable

§5.8

YOUR TURN

≈ 12 min

A 2-bit counter AB counts $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$ when $x = 1$ and holds when $x = 0$.

- (a) Write the state table.
- (b) Design it with D flip-flops.
- (c) Design it with T flip-flops. Which needs fewer gates?

Map variables in the order A, B, x .

Your turn — Solution

§5.8

SOLUTION

A B	x	A ⁺ B ⁺	T _A	T _B
0 0	0	0 0	0	0
0 0	1	0 1	0	1
0 1	0	0 1	0	0
0 1	1	1 0	1	1
1 0	0	1 0	0	0
1 0	1	1 1	0	1
1 1	0	1 1	0	0
1 1	1	0 0	1	1

- 1 (a) $x = 0$: next = present; $x = 1$: next = present + 1
- 2 (b) D: $D = Q^+$. B changes only when $x = 1 \rightarrow D_B = B \oplus x$
- 3 A changes when $x = 1$ and $B = 1 \rightarrow D_A = A \oplus Bx$
- 4 Check 01, $x = 1$: $D_B = 1 \oplus 1 = 0$, $D_A = 0 \oplus 1 = 1 \rightarrow 10 \checkmark$
- 5 (c) T: $T = Q \oplus Q^+$. The T_B column equals $x \rightarrow T_B = x$
- 6 $T_A = 1$ only on 01/1 and 11/1 $\rightarrow T_A = Bx$
- 7 T needs one AND gate; D needs two XORs and an AND
- 8 $x = 0$: every $T = 0 \rightarrow \text{hold} \checkmark$

Unused states

§5.8

The risk

Codes that are never assigned (e.g. 101, 110, 111 in a 5-state machine) were treated as don't-cares.

At power-up or after noise, the circuit may land in one.

The check

Evaluate the final equations from every unused code for every input. Each path must reach a valid state.

If one loops among unused codes, change a don't-care choice or add a reset.

Self-correcting

A circuit whose unused states all lead back into the valid sequence is self-correcting.

Counters in Lectures 21–22 are checked this way.

Five states need $\lceil \log_2 5 \rceil = 3$ flip-flops, leaving $8 - 5 = 3$ unused codes.

Concept check

1

When are two states equivalent?

2

Why can row matching miss an equivalence?

3

How many flip-flops does a 9-state machine need in binary? One-hot?

4

Why do JK designs often need fewer gates than D designs?

Key takeaways

Equivalent

same outputs, equivalent
next

Reduction

row match; implication table

Flip-flops

$$2^n \geq \text{states}$$

D design

$$D = Q^+$$

T design

$$T = Q \oplus Q^+$$

JK design

excitation table, X's

Diagram, table, maps, circuit, check: every time.

Homework #9 due at the start of Lecture 21

This afternoon's lab: Lab Hp3 — Calculator 3: magnitude comparator

Next: registers, shift registers and ripple counters